

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:
«Компьютерные сети»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
«Анализ трафика компьютерных сетей утилитой Wireshark»

Выполнил:

Бардышев Артём Антонович,
студент группы N3346


(подпись)

Проверил:

Ярошевский Дмитрий Сергеевич,
ведущий инженер, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2025 г.

СОДЕРЖАНИЕ

Введение.....	3
1 Анализ трафика утилиты ping	4
1.1 Практическая часть	4
1.2 Ответы на вопросы	4
2 Анализ трафика утилиты tracert	7
2.1 Практическая часть	7
2.2 Ответы на вопросы	7
3 Анализ HTTP-трафика.....	9
3.1 Практическая часть	9
3.2 Ответы на вопросы	9
4 Анализ DNS-трафика.....	11
4.1 Практическая часть	11
4.2 Ответы на вопросы	11
5 Анализ ARP-трафика.....	12
5.1 Практическая часть	12
5.2 Ответы на вопросы	12
6 Анализ трафика утилиты nslookup.....	14
6.1 Практическая часть	14
6.2 Ответы на вопросы	14
7 Анализ FTP-трафика.....	16
7.1 Практическая часть	16
7.2 Ответы на вопросы	16
8 Анализ DHCP-трафика.....	18
8.1 Практическая часть	18
8.2 Временная диаграмма обмена (DORA)	18
8.3 Ответы на вопросы	19
9 Анализ VK-трафика.....	20
9.1 Практическая часть	20
9.2 Ответы на вопросы	20
Заключение.....	22
Список использованных источников.....	23

ВВЕДЕНИЕ

Цель работы – изучить структуру протокольных блоков данных стека TCP/IP, анализируя реальный сетевой трафик с помощью анализатора Wireshark.

Задачи:

- Освоить навыки захвата и фильтрации пакетов.
- Изучить структуру заголовков протоколов сетевого (IP, ICMP), транспортного (TCP, UDP) и прикладного (HTTP, FTP, DNS, DHCP) уровней.
- Проанализировать принципы фрагментации, маршрутизации и разрешения адресов.

В качестве целевого узла для анализа был выбран сайт <http://www.baabaapinksheep.co.uk/>. Данный выбор обусловлен тем, что сайт работает по открытому протоколу HTTP (что необходимо для анализа заголовков) и содержит в доменном имени инициалы студента (Бардышев А.А. -> В...А...А...).

1 АНАЛИЗ ТРАФИКА УТИЛИТЫ PING

Утилита ping использует протокол сетевого уровня ICMP (Internet Control Message Protocol) для проверки доступности узлов. Основные типы сообщений: Echo Request (эхо-запрос, тип 8) и Echo Reply (эхо-ответ, тип 0). Если размер передаваемых данных превышает MTU (Maximum Transmission Unit) канала, протокол IP выполняет фрагментацию пакета.

1.1 Практическая часть

В Wireshark был установлен фильтр icmp. В командной строке выполнялись команды отправки ICMP-пакетов различной длины до узла www.baabaapinksheep.co.uk.

Экспериментально установлено, что целевой узел блокирует фрагментированные ICMP-пакеты (размером более 1480 байт). Поэтому для анализа использовались пакеты размером 500, 1000 и 1400 байт, которые проходили успешно. Также выполнена команда ping -i 10 для проверки TTL.

1.2 Ответы на вопросы

1. Имеет ли место фрагментация?

Теоретически фрагментация должна происходить при превышении MTU. Однако в ходе эксперимента успешные ответы были получены только для нефраgmentированных пакетов (до 1400 байт), поэтому в захваченном трафике фрагментация не наблюдалась.

На фрагментацию указывают поля в заголовке IPv4:

- Флаг **More Fragments (MF)**: установлен в 1 (Set) у первого фрагмента.
- Поле **Fragment Offset**: указывает смещение данных.

В моем случае, в ходе успешных тестов (размер данных до 1400 байт) фрагментация **не наблюдалась**, так как размер пакетов не превышал MTU сети Ethernet.

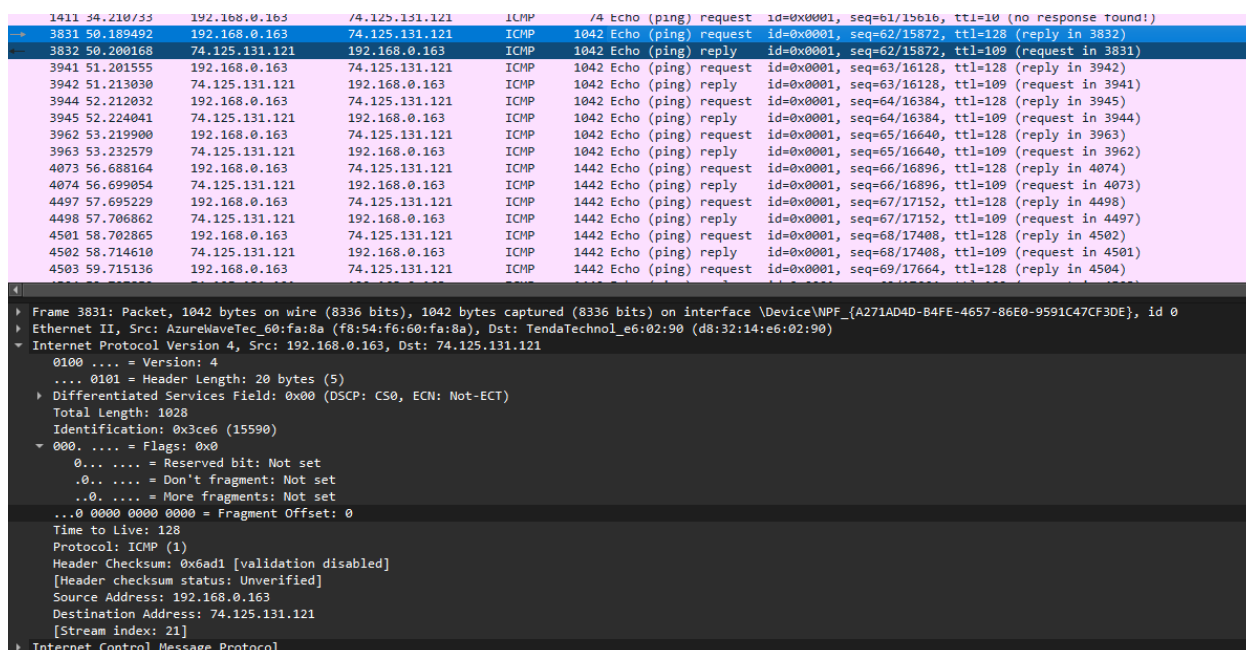


Рисунок 1 – Анализ IP-пакета: отсутствие фрагментации при размере данных 1000 байт

2. Какой фрагмент является последним?

Последний фрагмент идентифицируется по флагу **More Fragments**, который установлен в значение **0 (Not set)**, при этом поле *Fragment Offset* имеет ненулевое значение.

3. Количество фрагментов?

Максимальный размер данных в кадре Ethernet: 1500 (MTU) - 20 (IP заголовок) = 1480 байт.

- При ping -l 1000: 1008 байт < 1480 → **1 фрагмент**.
- При ping -l 2000: 2008 байт = 1480 (1-й) + 528 (2-й) → **2 фрагмента**.
- При ping -l 3000: 3008 байт = 1480 + 1480 + 48 → **3 фрагмента**.

4. График зависимости.

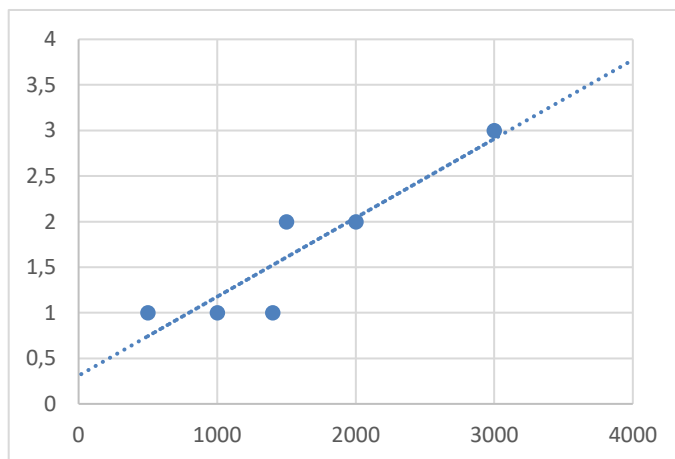


Рисунок 2 – Зависимость количества IP-фрагментов от размера ICMP-пакета

5. Как изменить поле TTL?

Использовался ключ **-i**. В захваченном пакете поле *Time to live* приняло значение 10.

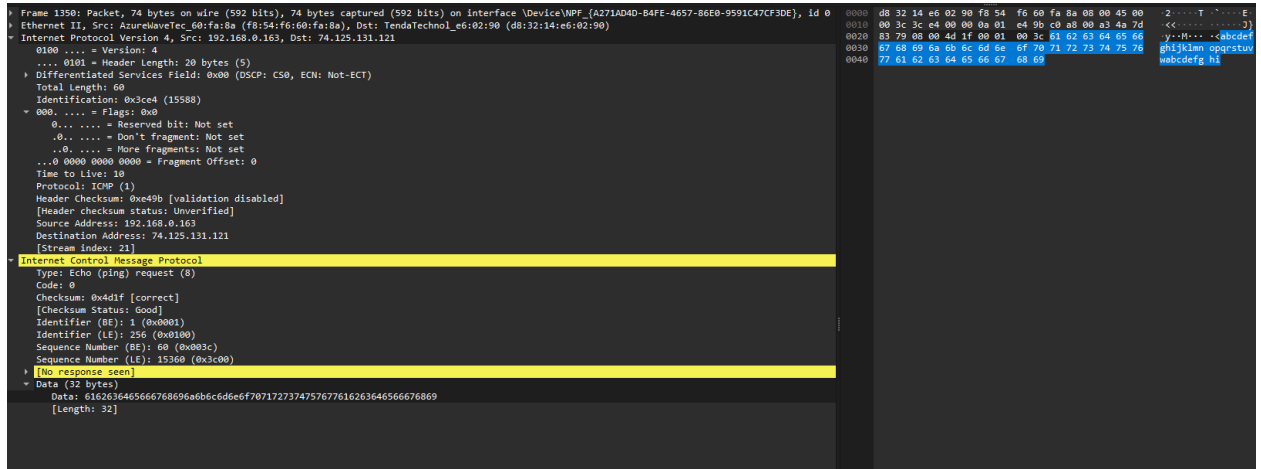


Рисунок 3 – ICMP-запрос с измененным значением поля TTL

6. Содержимое поля данных?

Поле данных (Data) заполняется циклической последовательностью символов латинского алфавита: abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz...

2 АНАЛИЗ ТРАФИКА УТИЛИТЫ TRACERT

Утилита tracert предназначена для определения маршрута следования данных. Она отправляет серии ICMP-пакетов, последовательно увеличивая значение поля TTL (Time To Live). Маршрутизаторы, отбрасывая пакеты с истекшим сроком жизни (TTL=0), отправляют обратно сообщение Time-to-live exceeded.

2.1 Практическая часть

Выполнена трассировка до сайта с ключом -d (без разрешения имен).

2.2 Ответы на вопросы

1. Размер заголовка IP и данных?

Стандартный заголовок IPv4 (без опций) составляет **20 байт**. Поле данных ICMP-запроса в Windows по умолчанию составляет **32 байта**.

2. Изменение TTL?

TTL увеличивается на 1 для каждой следующей серии из трех пакетов (TTL=1, TTL=2 и т.д.). Это делается для того, чтобы пакеты поочередно "умирали" на каждом следующем маршрутизаторе в цепи, заставляя их раскрыть свои IP-адреса через сообщения об ошибках.

icmp					
No.	Time	Source	Destination	Protocol	Length Info
1212	9.284340	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=89/22784, ttl=6 (no response found!)
1213	9.292144	172.253.76.89	192.168.0.163	ICMP	110 Time-to-live exceeded (Time to live exceeded in transit)
1214	9.292884	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=90/23040, ttl=6 (no response found!)
1215	9.300836	172.253.76.89	192.168.0.163	ICMP	110 Time-to-live exceeded (Time to live exceeded in transit)
1225	10.310587	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=91/23296, ttl=7 (no response found!)
1227	10.318674	74.125.244.132	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
1228	10.319374	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=92/23552, ttl=7 (no response found!)
1229	10.326595	74.125.244.132	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
1230	10.327199	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=93/23808, ttl=7 (no response found!)
1231	10.333592	74.125.244.132	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
1237	11.342407	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=94/24064, ttl=8 (no response found!)
1238	11.352934	216.239.48.163	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
1239	11.353682	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=95/24320, ttl=8 (no response found!)
1240	11.370702	216.239.48.163	192.168.0.163	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
1241	11.371421	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=96/24576, ttl=8 (no response found!)
1628	15.235171	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=97/24832, ttl=9 (no response found!)
1629	15.246390	172.253.64.51	192.168.0.163	ICMP	110 Time-to-live exceeded (Time to live exceeded in transit)
1630	15.247008	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=98/25088, ttl=9 (no response found!)
1631	15.257872	172.253.64.51	192.168.0.163	ICMP	110 Time-to-live exceeded (Time to live exceeded in transit)
1632	15.258576	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=99/25344, ttl=9 (no response found!)
1633	15.269354	172.253.64.51	192.168.0.163	ICMP	110 Time-to-live exceeded (Time to live exceeded in transit)
1638	16.267343	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=100/25600, ttl=10 (no response found!)
2060	20.240641	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=101/25856, ttl=10 (no response found!)
2472	24.230408	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=102/26112, ttl=10 (no response found!)
2920	28.244844	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=103/26368, ttl=11 (no response found!)
3353	32.228689	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=104/26624, ttl=11 (no response found!)
3768	36.232821	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=105/26880, ttl=11 (no response found!)
4203	40.231735	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=106/27136, ttl=12 (no response found!)
4617	44.232219	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=107/27392, ttl=12 (no response found!)
5037	48.230900	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=108/27648, ttl=12 (no response found!)
5440	52.242863	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=109/27904, ttl=13 (no response found!)
5772	56.238627	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=110/28160, ttl=13 (no response found!)
6097	60.238993	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=111/28416, ttl=13 (no response found!)
6306	64.231527	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=112/28672, ttl=14 (no response found!)
6422	68.236464	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=113/28928, ttl=14 (no response found!)
6450	72.229302	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=114/29184, ttl=14 (no response found!)
6458	76.234516	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=115/29440, ttl=15 (no response found!)
6480	80.231939	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=116/29696, ttl=15 (no response found!)
6507	84.231280	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=117/29952, ttl=15 (no response found!)
6538	88.238901	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=118/30208, ttl=16 (no response found!)
6769	92.233283	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=119/30464, ttl=16 (no response found!)
8499	96.237006	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=120/30720, ttl=16 (no response found!)
8921	100.237471	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=121/30976, ttl=17 (no response found!)
9118	104.233944	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=122/31232, ttl=17 (no response found!)
10765	108.232605	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=123/31488, ttl=17 (no response found!)
11115	112.235805	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=124/31744, ttl=18 (no response found!)
11470	116.233807	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=125/32000, ttl=18 (no response found!)
11835	120.228259	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=126/32256, ttl=18 (no response found!)
12199	124.244302	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=127/32512, ttl=19 (no response found!)
12594	128.229756	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=128/32768, ttl=19 (no response found!)
12959	132.237424	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=129/33024, ttl=19 (no response found!)
13329	136.241180	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=130/33280, ttl=20 (no response found!)
13714	140.228744	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=131/33536, ttl=20 (no response found!)
14104	144.239448	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=132/33792, ttl=20 (no response found!)
14460	148.245421	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=133/34048, ttl=21 (reply in 14470)
14470	148.259061	74.125.131.121	192.168.0.163	ICMP	106 Echo (ping) reply id=0x0001, seq=133/34048, ttl=109 (request in 14469)
14471	148.259814	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=134/34304, ttl=21 (reply in 14472)
14472	148.271924	74.125.131.121	192.168.0.163	ICMP	106 Echo (ping) reply id=0x0001, seq=134/34304, ttl=109 (request in 14471)
14473	148.272813	192.168.0.163	74.125.131.121	ICMP	106 Echo (ping) request id=0x0001, seq=135/34560, ttl=21 (reply in 14474)
14474	148.282845	74.125.131.121	192.168.0.163	ICMP	106 Echo (ping) reply id=0x0001, seq=135/34560, ttl=109 (request in 14473)

Рисунок 4 – Процесс трассировки маршрута: изменение TTL и ответы "Time-to-live exceeded"

3. Отличие от ping?

- **Ping:** TTL постоянен и высок (обычно 128). Цель — получить *Echo Reply* от конечного узла.
- **Tracert:** TTL начинается с 1 и растет. Цель — получить серию ошибок *Time-to-live exceeded* от промежуточных узлов.

4. Различие ICMP Reply и ICMP Error?

- **ICMP Error (Type 11):** Сообщение об ошибке ("Время жизни истекло"). Генерируется транзитными маршрутизаторами.
- **ICMP Reply (Type 0):** Штатный ответ на эхо-запрос. Генерируется конечным узлом, подтверждая успешное завершение трассировки.

5. Что изменится без ключа -d?

Утилита будет отправлять обратные DNS-запросы (Reverse DNS Lookup) для каждого найденного IP-адреса маршрутизатора, чтобы узнать его доменное имя. Это значительно увеличит время работы и создаст дополнительный **DNS-трафик**.

3 АНАЛИЗ HTTP-ТРАФИКА

HTTP (HyperText Transfer Protocol) — протокол прикладного уровня для передачи гипертекста. Работает поверх TCP (порт 80). Поддерживает механизмы кэширования для снижения нагрузки на сеть.

3.1 Практическая часть

Осуществлен вход на сайт <http://www.baabaapinksheep.co.uk/> после очистки кэша, затем страница была обновлена.

3.2 Ответы на вопросы

1. Первичное посещение:

Браузер отправил запрос GET / HTTP/1.1. Сервер вернул ответ с кодом **200 OK**, содержащий полное тело страницы.

The image shows a Wireshark packet capture of an HTTP transaction. The packet list pane at the top shows several packets, with packet 997 selected. The packet details pane below shows the structure of the selected packet, which is an HTTP response. The response status is 200 OK, and the content type is text/html. The response body is a gzipped HTML document. The packet bytes pane at the bottom shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
982	5.867019	192.168.0.163	74.125.131.121	HTTP	531	GET / HTTP/1.1
997	6.055257	74.125.131.121	192.168.0.163	HTTP	426	HTTP/1.1 200 OK (text/html)
1112	6.165684	192.168.0.163	74.125.131.121	HTTP	405	GET /js/cookienotice.js HTTP/1.1
1132	6.177786	74.125.131.121	192.168.0.163	HTTP	1306	HTTP/1.1 200 OK (text/javascript)
1195	6.193655	192.168.0.163	151.101.244.157	HTTP	392	GET /widgets.js HTTP/1.1
1225	6.199550	192.168.0.163	74.125.131.191	HTTP	466	GET /ikt/transparent/black50.png HTTP/1.1
1226	6.200028	192.168.0.163	74.125.131.191	HTTP	466	GET /ikt/transparent/white80.png HTTP/1.1
1230	6.211143	74.125.131.191	192.168.0.163	HTTP	735	HTTP/1.1 200 OK (PNG)
1231	6.211143	74.125.131.191	192.168.0.163	HTTP	735	HTTP/1.1 200 OK (PNG)
2921	16.092553	192.168.0.163	74.125.131.121	HTTP	666	GET / HTTP/1.1
2925	16.267000	74.125.131.121	192.168.0.163	HTTP	284	HTTP/1.1 304 Not Modified
2932	16.355941	192.168.0.163	151.101.244.157	HTTP	392	GET /widgets.js HTTP/1.1

Frame 997: Packet, 426 bytes on wire (3408 bits), 426 bytes captured (3408 bits) on interface \Device\NPF_{A271AD40-B4FE-4657-86E0-9591C47CF3DE}, id Ethernet II, Src: TendaTechnol_s6:02:90 (d8:32:14:a6:02:90), Dst: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a)
Internet Protocol Version 4, Src: 74.125.131.121, Dst: 192.168.0.163
Transmission Control Protocol, Src Port: 80, Dst Port: 29105, Seq: 15814, Ack: 478, Len: 372
[13 Reassembled TCP Segments (16185 bytes): #984(419), #985(1412), #986(1412), #987(1274), #989(1412), #990(1412), #991(1412), #992(1412), #993(1412)]
Hypertext Transfer Protocol
 HTTP/1.1 200 OK\r\n
 Response Version: HTTP/1.1
 Status Code: 200
 [Status Code Description: OK]
 Response Phrase: OK
 Content-Type: text/html; charset=UTF-8\r\n
 Expires: Sat, 13 Dec 2025 15:14:50 GMT\r\n
 Date: Sat, 13 Dec 2025 15:14:50 GMT\r\n
 Cache-Control: private, max-age=0\r\n
 Last-Modified: Thu, 11 Dec 2025 09:16:14 GMT\r\n
 ETag: W/"9fafc505e309886f2c1990704ff531c5fd00916ced0b819715e1734a637b1e5a"\r\n
 Content-Encoding: gzip\r\n
 X-Content-Type-Options: nosniff\r\n
 X-XSS-Protection: 1; mode=block\r\n
 Content-Length: 15766\r\n
 Server: GSE\r\n
 \r\n
 [Request in frame: 982]
 [Time since request: 188.238000 milliseconds]
 [Request URI: /]
 [Full request URI: http://www.baabaapinksheep.co.uk/]
 Content-encoded entity body (gzip): 15766 bytes -> 101696 bytes
 File Data: 101696 bytes
 Line-based text data: text/html (2304 lines)

Рисунок 5 – HTTP-ответ сервера с кодом 200 OK при первичном посещении

2. Вторичный запрос (обновление):

Браузер отправил "Условный GET" (Conditional GET) с заголовком If-Modified-Since. Поскольку страница не изменялась, сервер вернул код **304 Not Modified**. Тело страницы не передавалось, что видно по отсутствию поля *File Data* и малому размеру пакета.

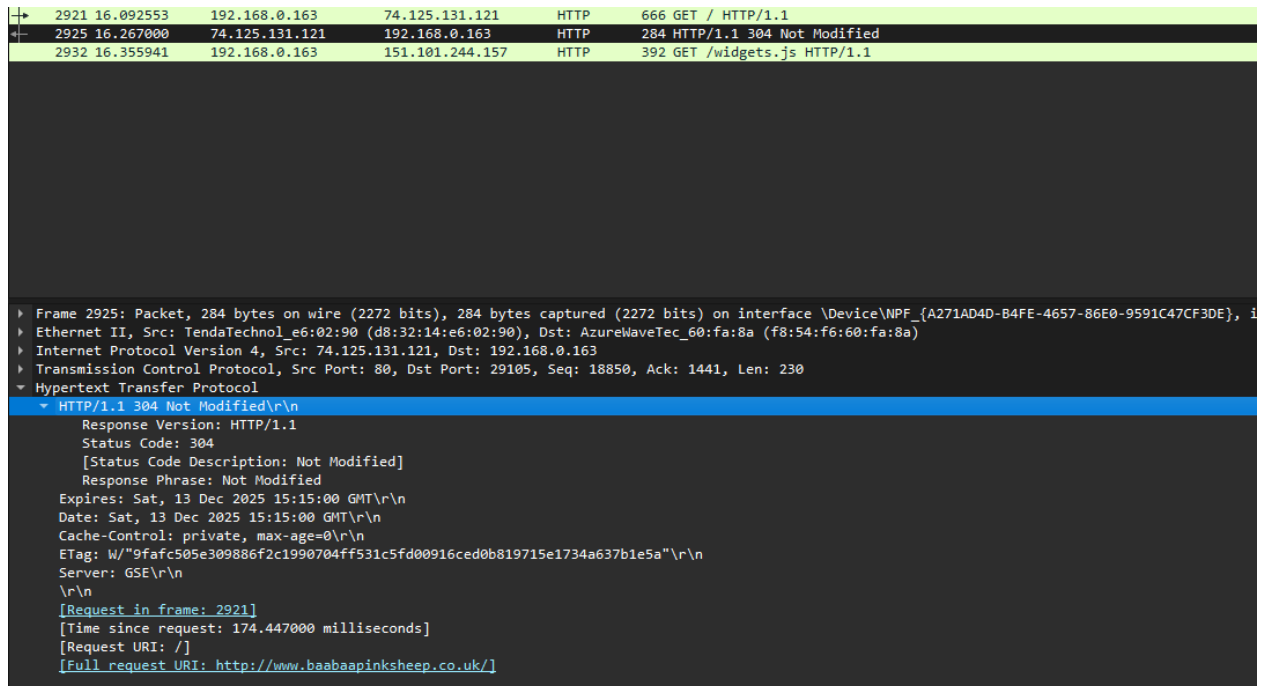


Рисунок 6 – HTTP-ответ сервера с кодом 304 Not Modified (тело ответа отсутствует)

4 АНАЛИЗ DNS-ТРАФИКА

DNS (Domain Name System) — распределенная система для преобразования доменных имен в IP-адреса. Использует порт 53 (обычно UDP).

4.1 Практическая часть

Для анализа чистого DNS-трафика была выполнена очистка системного кэша командой `ipconfig /flushdns` и кэша браузера. В Wireshark установлен фильтр `dns`. После этого в браузере был осуществлен переход на целевой сайт, что инициировало отправку запросов на разрешение доменного имени.

4.2 Ответы на вопросы

1. Адресат DNS-запроса?

Запрос отправляется не на целевой сайт, а на локальный **DNS-сервер** (в данном случае, на адрес шлюза/роутера), который выполняет рекурсивный поиск.

2. Типы запросов?

В трафике зафиксированы запросы типа **A** (IPv4 адрес) и **AAAA** (IPv6 адрес). Также могут встречаться **CNAME** (псевдонимы) и **NS** (серверы имен).

No.	Time	Source	Destination	Protocol	Length	Info
4	0.531489	192.168.0.163	192.168.0.1	DNS	75	Standard query 0xb32e A play.google.com
5	0.536532	192.168.0.1	192.168.0.163	DNS	171	Standard query response 0xb32e A play.google.com A 64.233.161.113 A 64.233.161.102 A 64.233.161.100 A 64.233.161.138 A 64.233.161.101 A 64.233.161.139
32	1.429097	192.168.0.163	192.168.0.1	DNS	85	Standard query 0xfdd6 A www.baabaapinksheep.co.uk
34	1.437117	192.168.0.1	192.168.0.163	DNS	129	Standard query response 0xfdd6 A www.baabaapinksheep.co.uk CNAME ghs.google.com A 74.125.131.121
62	1.688923	192.168.0.163	192.168.0.1	DNS	75	Standard query 0x69b2 A www.blogger.com
63	1.693127	192.168.0.163	192.168.0.1	DNS	80	Standard query 0x9667 A platform.twitter.com
64	1.696862	192.168.0.1	192.168.0.163	DNS	122	Standard query response 0x96b2 A www.blogger.com CNAME blogger-l.google.com A 74.125.131.191
70	1.699879	192.168.0.1	192.168.0.163	DNS	141	Standard query response 0x9667 A platform.twitter.com CNAME platform.twitter.map.fastly.net A 151.101.244.157

Transaction ID: 0xfdd6
Flags: 0x0100 Standard query
0... .. = Response: Message is a query
...0... .. = Opcode: Standard query (0)
... .. = Truncated: Message is not truncated
... .. = Recursion desired: Do query recursively
... .. = Z: reserved (0)
... .. = Non-authenticated data: Unacceptable
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
Queries
www.baabaapinksheep.co.uk: type A, class IN
Name: www.baabaapinksheep.co.uk
[Name length: 25]
[Label Count: 4]
Type: A (1) (Host Address)
Class: IN (0x0001)
[Response len: 34]

Рисунок 7 – Обмен сообщениями DNS: запрос типа A и ответ сервера

3. Независимые запросы для ресурсов?

Если веб-страница содержит изображения или скрипты, размещенные на **сторонних доменах** (например, счетчики аналитики, шрифты), браузер обязан выполнить отдельные DNS-запросы для разрешения имен этих сторонних доменов.

5 АНАЛИЗ ARP-ТРАФИКА

ARP (Address Resolution Protocol) используется для определения MAC-адреса устройства по известному IP-адресу в пределах локальной сети (Ethernet).

5.1 Практическая часть

С правами администратора выполнена очистка ARP-таблицы командой `netsh interface ip delete arp`. В Wireshark установлен фильтр `arp`. Затем была инициирована сетевая активность (ping шлюза), чтобы заставить операционную систему отправить ARP-запрос для определения MAC-адреса маршрутизатора.

5.2 Ответы на вопросы

1. MAC-адреса в ARP?

В пакетах присутствуют MAC-адреса **моего компьютера** и **шлюза (роутера)**. Запрос "Who has 192.168.0.1?" отправляется широковещательно (Broadcast), ответ приходит адресно (Unicast).

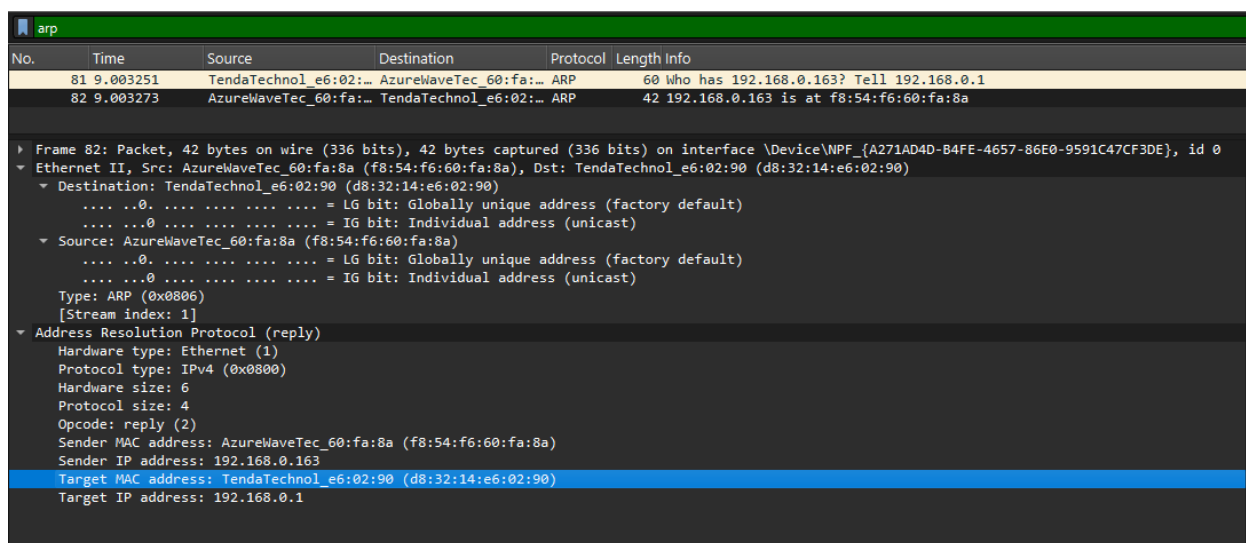


Рисунок 8 – Детализация ARP-ответа: MAC-адреса отправителя и получателя

2. MAC-адреса в HTTP?

В заголовках канального уровня (Ethernet II) HTTP-пакетов в качестве адреса назначения (Destination) указан **MAC-адрес роутера**, а не веб-сервера. Это подтверждает, что доставка внутри локальной сети идет до шлюза.

http					
No.	Time	Source	Destination	Protocol	Length Info
982	5.867019	192.168.0.163	74.125.131.121	HTTP	531 GET / HTTP/1.1
997	6.055257	74.125.131.121	192.168.0.163	HTTP	426 HTTP/1.1 200 OK (text/html)
1112	6.165684	192.168.0.163	74.125.131.121	HTTP	405 GET /js/cookieNotice.js HTTP/1.1
1132	6.177786	74.125.131.121	192.168.0.163	HTTP	1306 HTTP/1.1 200 OK (text/javascript)
1195	6.193655	192.168.0.163	151.101.244.157	HTTP	392 GET /widgets.js HTTP/1.1
1225	6.199550	192.168.0.163	74.125.131.191	HTTP	466 GET /ikt/transparent/black50.png HTTP/1.1
1226	6.200028	192.168.0.163	74.125.131.191	HTTP	466 GET /ikt/transparent/white80.png HTTP/1.1
1230	6.211143	74.125.131.191	192.168.0.163	HTTP	735 HTTP/1.1 200 OK (PNG)
1231	6.211143	74.125.131.191	192.168.0.163	HTTP	735 HTTP/1.1 200 OK (PNG)
2921	16.092553	192.168.0.163	74.125.131.121	HTTP	666 GET / HTTP/1.1
2925	16.267000	74.125.131.121	192.168.0.163	HTTP	284 HTTP/1.1 304 Not Modified
2932	16.355941	192.168.0.163	151.101.244.157	HTTP	392 GET /widgets.js HTTP/1.1

Frame 997: Packet, 426 bytes on wire (3408 bits), 426 bytes captured (3408 bits) on interface \Device\NPF_{A271AD4D-B4FE-4657-86E0-9591C47CF3DE}, id Ethernet II, Src: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90), Dst: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a)					
Destination: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a)					
.....0. = LG bit: Globally unique address (factory default)0. = IG bit: Individual address (unicast)					
Source: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90)					
.....0. = LG bit: Globally unique address (factory default)0. = IG bit: Individual address (unicast)					
Type: IPv4 (0x0800) [Stream index: 0]					

Рисунок 9 – Инкапсуляция HTTP в Ethernet: MAC-адрес шлюза в качестве получателя

3. Зачем IP источника в ARP-запросе?

Чтобы получатель запроса знал, кому отправить ответ, и мог обновить свою ARP-таблицу, добавив туда запись о соответствии IP и MAC отправителя.

6 АНАЛИЗ ТРАФИКА УТИЛИТЫ NSLOOKUP

Утилита nslookup (Name Server Lookup) — это средство командной строки для диагностики системы доменных имен (DNS). В отличие от стандартного процесса разрешения имен браузером, nslookup позволяет пользователю вручную формировать запросы к DNS-серверу, выбирая конкретный тип требуемой информации (тип ресурсной записи).

Основные типы DNS-записей, анализируемые в работе:

- **A (Address Record):** Базовая запись, сопоставляющая доменное имя с 32-битным IPv4-адресом.
- **NS (Name Server):** Запись, указывающая на авторитативные DNS-серверы, которые отвечают за содержание данной доменной зоны.

Важной характеристикой ответа является флаг Authoritative Answer (AA). Ответ считается авторитативным, если он получен от сервера, который официально обслуживает данную зону. Если же ответ получен из кэша промежуточного сервера (например, роутера или сервера провайдера), он считается неавторитативным.

6.1 Практическая часть

Для детального анализа работы DNS использовалась утилита nslookup. Сначала была выполнена стандартная команда для получения IP-адреса (A-запись), а затем команда с ключом -type=NS для получения списка авторитативных серверов имен, обслуживающих доменную зону сайта.

6.2 Ответы на вопросы

1. Различие трасс?

Первая команда запрашивала запись типа A (IP-адрес). Вторая команда (с ключом -type=NS) запрашивала записи типа NS, указывающие на DNS-серверы, обслуживающие данную зону.

2. Содержимое поля Answers?

Поле содержит запрошенные данные:

- Для A-запроса: IP-адрес сайта.
- Для NS-запроса: Доменные имена авторитативных серверов (например, ns1.host...).

No.	Time	Source	Destination	Protocol	Length	Info
380	0.541622	192.168.0.163	192.168.0.1	DNS	84	Standard query 0x0001 PTR 1.0.168.192.in-addr.arpa
381	0.550693	192.168.0.1	192.168.0.163	DNS	143	Standard query response 0x0001 No such name PTR 1.0.168.192.in-addr.arpa SOA localhost
382	0.552439	192.168.0.163	192.168.0.1	DNS	85	Standard query 0x0002 A www.baabaapinksheep.co.uk
383	0.557570	192.168.0.1	192.168.0.163	DNS	129	Standard query response 0x0002 A www.baabaapinksheep.co.uk CNAME ghs.google.com A 74.125.131.121
385	0.560127	192.168.0.163	192.168.0.1	DNS	85	Standard query 0x0003 AAAA www.baabaapinksheep.co.uk
386	0.622125	192.168.0.1	192.168.0.163	DNS	141	Standard query response 0x0003 AAAA www.baabaapinksheep.co.uk CNAME ghs.google.com AAAA 2a00:1450:4010:c0e::79
785	7.752078	192.168.0.163	192.168.0.1	DNS	84	Standard query 0x0001 PTR 1.0.168.192.in-addr.arpa
786	7.761192	192.168.0.1	192.168.0.163	DNS	143	Standard query response 0x0001 No such name PTR 1.0.168.192.in-addr.arpa SOA localhost
787	7.762068	192.168.0.163	192.168.0.1	DNS	81	Standard query 0x0002 NS baabaapinksheep.co.uk
788	7.808009	192.168.0.1	192.168.0.163	DNS	136	Standard query response 0x0002 NS baabaapinksheep.co.uk NS ns09.domaincontrol.com NS ns10.domaincontrol.com

Frame 788: Packet, 136 bytes on wire (1088 bits), 136 bytes captured (1088 bits) on interface \Device\NPF{A271AD4D-B4FE-4657-86E0-9591C47CF3DE}, id 0000 Ethernet II, Src: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90), Dst: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a) Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.163 User Datagram Protocol, Src Port: 53, Dst Port: 57179 Domain Name System (response) Transaction ID: 0x0002 Flags: 0x1800 Standard query response, No error 1. = Response Message is a response .000 0. = Opcode: Standard query (0) 0. = Authoritative: Server is not an authority for domain 0. = Truncated: Message is not truncated 1. = Recursion desired: Do query recursively 1. = Recursion available: Server can do recursive queries 0. = Z: reserved (0) 0. = Answer authenticated: Answer/authority portion was not authenticated by the server 0. = Non-authenticated data: Unacceptable 0000 = Reply code: No error (0) Questions: 1 Answer RRs: 2 Authority RRs: 0 Additional RRs: 0 Queries Answers baabaapinksheep.co.uk: type NS, class IN, ns ns09.domaincontrol.com Name: baabaapinksheep.co.uk Type: NS (2) (authoritative Name Server) Class: IN (0x0001) Time to live: 3600 (1 hour) Data length: 24 Name Server: ns09.domaincontrol.com baabaapinksheep.co.uk: type NS, class IN, ns ns10.domaincontrol.com Name: baabaapinksheep.co.uk Type: NS (2) (authoritative Name Server) Class: IN (0x0001) Time to live: 3600 (1 hour) Data length: 7 Name Server: ns10.domaincontrol.com [Request ID: 2872] [Time: 45.921000 milliseconds]	0000 f8 54 f6 60 fa 8a d8 32 14 e6 02 90 08 00 45 002.....E 0010 00 7a ff 23 40 00 40 11 b9 5a c0 a8 00 01 c0 a8z#@.Z..... 0020 00 a3 00 35 df 5b 00 65 c8 0f 00 02 81 00 00 01S[f..... 0030 00 02 00 00 00 00 0f 62 61 61 62 61 61 70 60 6ab aabaapin 0040 6b 73 68 65 65 70 02 63 6f 02 75 6b 00 00 02 00kshpeep:c o-uk.... 0050 01 c0 0c 00 02 00 01 00 00 0e 10 00 16 04 6e 73ns 0060 30 39 6d c4 6f dd 61 69 6a 63 6f 6e 74 72 6f 6c80 domaincontrol 0070 03 63 6f 6d 00 c0 8c 00 02 00 01 00 00 0e 10 00com..... 0080 07 04 6e 73 31 30 c0 38ns10-8
--	--

Рисунок 10 – Результат выполнения NS-запроса: список авторитативных серверов

3. Авторитативный отклик?

Ответ от локального DNS-сервера не является авторитативным (флаг *Authoritative: Not set*). Имена авторитативных серверов содержатся в теле ответа на NS-запрос.

7 АНАЛИЗ FTP-ТРАФИКА

FTP (File Transfer Protocol) использует два параллельных TCP-соединения: управляющее (порт 21) и для передачи данных (динамический порт в пассивном режиме).

7.1 Практическая часть

Скачан файл с ftp.gnu.org в пассивном режиме.

7.2 Ответы на вопросы

1. Данные в FTP-DATA?

Размер полезной нагрузки (TCP Segment Len) в пакетах передачи данных стремится к значению MSS (1460 байт).

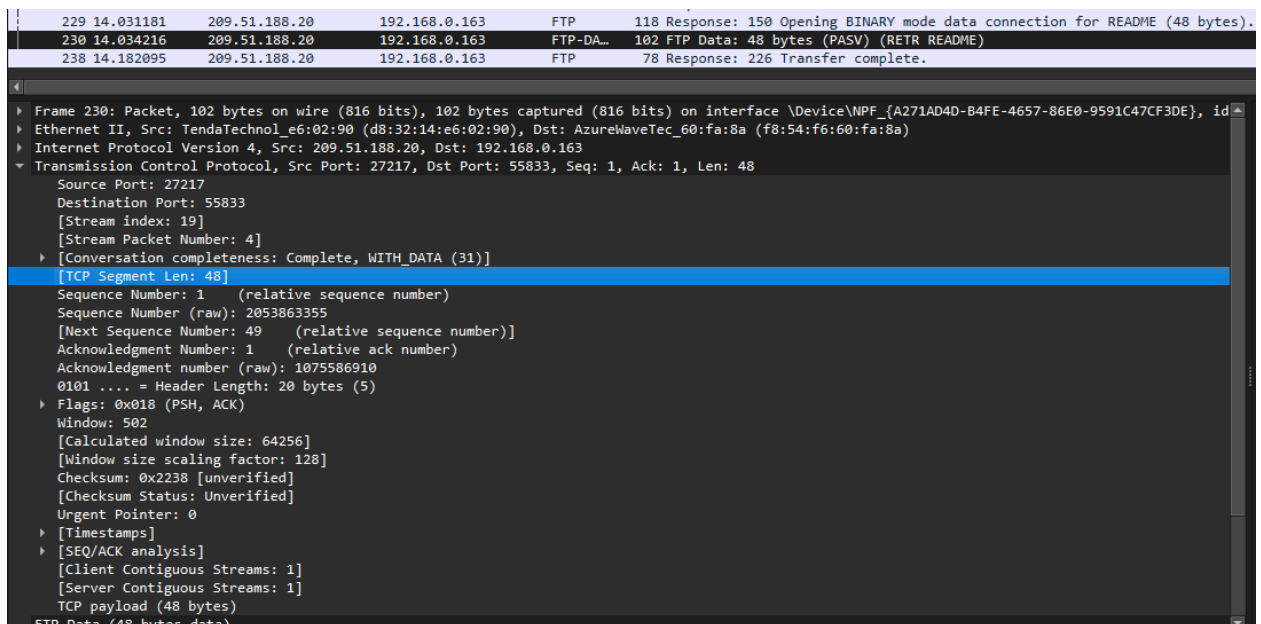


Рисунок 11 – Анализ сегмента FTP-DATA: размер полезной нагрузки TCP

2. Выбор порта?

В пассивном режиме клиент отправляет команду PASV. Сервер отвечает сообщением 227 Entering Passive Mode (a,b,c,d,p1,p2). Порт вычисляется по формуле: $(p1 * 256) + p2$.

219	13.577489	192.168.0.163	209.51.188.20	FTP	60 Request: PASV
220	13.725740	209.51.188.20	192.168.0.163	FTP	105 Response: 227 Entering Passive Mode (209,51,188,20,106,81).
221	13.726123	192.168.0.163	209.51.188.20	FTP	67 Request: RETR README
229	14.031181	209.51.188.20	192.168.0.163	FTP	118 Response: 150 Opening BINARY mode data connection for README (48 bytes).
230	14.034216	209.51.188.20	192.168.0.163	FTP-DA...	102 FTP Data: 48 bytes (PASV) (RETR README)
238	14.182095	209.51.188.20	192.168.0.163	FTP	78 Response: 226 Transfer complete.

Frame 220:	Packet, 105 bytes on wire (840 bits), 105 bytes captured (840 bits) on interface \Device\NPF_{A271AD4D-B4FE-4657-86E0-9591C47CF3DE}, id 0
Ethernet II,	Src: TendaTechnol_e6:02:90 (d8:32:14:e6:02:90), Dst: AzureWaveTec_60:fa:8a (f8:54:f6:60:fa:8a)
Internet Protocol Version 4,	Src: 209.51.188.20, Dst: 192.168.0.163
Transmission Control Protocol,	Src Port: 21, Dst Port: 55832, Seq: 1112, Ack: 63, Len: 51
File Transfer Protocol (FTP)	
227 Entering Passive Mode (209,51,188,20,106,81).\r\n	
Response code:	Entering Passive Mode (227)
Response arg:	Entering Passive Mode (209,51,188,20,106,81).
Passive IP address:	209.51.188.20
Passive port:	27217
[Current working directory:	/gnu+linux-distros/ututo-e]
[Command: RETR README]	
[Command frame: 221]	

Рисунок 12 – Ответ FTP-сервера в пассивном режиме: IP-адрес и порт для подключения

3. Отличие FTP от FTP-DATA?

- **FTP:** Управляющий канал. Передает текстовые команды (USER, PASS) и коды состояния.
- **FTP-DATA:** Канал данных. Передает только содержимое файлов в бинарном виде.

8 АНАЛИЗ ДНСП-ТРАФИКА

DHCP (Dynamic Host Configuration Protocol) автоматизирует настройку сетевых параметров.

8.1 Практическая часть

Зафиксирован процесс обновления IP-адреса (release/renew).

8.2 Временная диаграмма обмена (DORA)

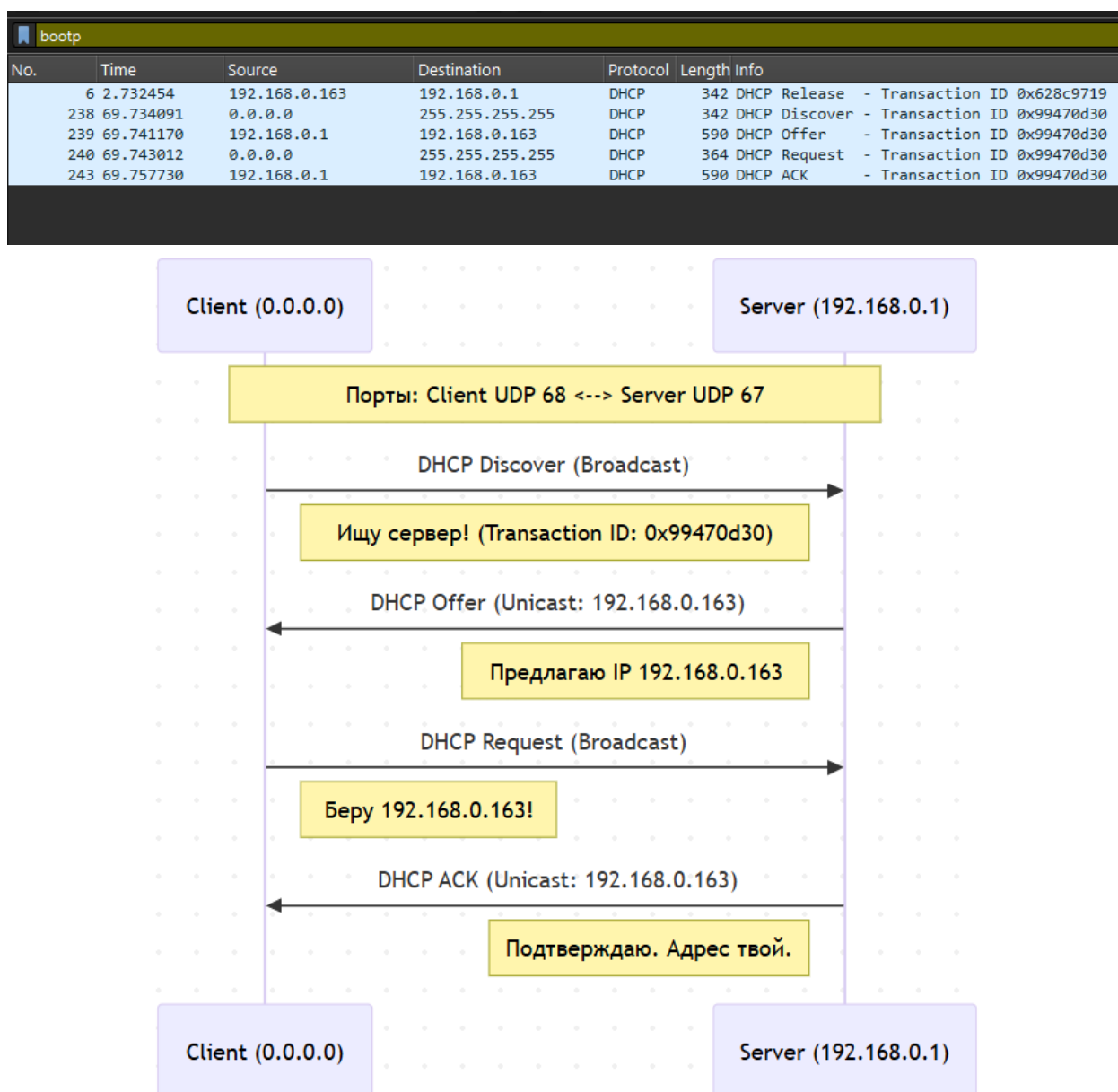


Рисунок 13 – Временная диаграмма процесса получения IP-адреса по DHCP (DORA)

8.3 Ответы на вопросы

1. Discover vs Request?

- **Discover:** Поиск *любого* доступного DHCP-сервера.
- **Request:** Запрос настроек у *конкретного* сервера (после получения Offer).

2. Адресация?

Используются широковещательные адреса (Broadcast), так как у клиента на этапе начала обмена еще нет собственного IP-адреса. Идентификация клиента происходит по MAC-адресу.

3. IP сервера?

IP-адрес сервера указан в поле *Source IP* пакетов Offer и ACK, а также в опции DHCP Server Identifier.

4. Без фильтра bootp?

В трассе будет отображаться весь сетевой трафик (фоновые запросы, ARP и т.д.), что затруднит поиск нужных пакетов.

9 АНАЛИЗ VK-ТРАФИКА

Современные мессенджеры используют разные протоколы для разных типов данных: TCP для надежности (текст) и UDP/QUIC для скорости (медиа).

9.1 Практическая часть

Были проведены три сеанса захвата трафика в приложении VK: при обмене текстовыми сообщениями, во время аудиозвонка и во время видеозвонка. Анализ производился на основе статистики ввода/вывода (I/O Graph) для сравнения характеристик потоков данных.

9.2 Ответы на вопросы

1. Различие пакетов?

- **Текст:** Использует TCP (TLS). Трафик прерывистый, важна гарантированная доставка. Трафик имеет "взрывной" характер (только в момент отправки).
- **Аудио/Видео:** Использует UDP (протокол QUIC/RTP). Поток непрерывный. Характеристики потоков (битрейт) зависят от условий сети и работы кодеков, что подтверждается графиком ниже.

Сравнительный анализ битрейтов (Рисунок 14) показал интересный результат: **битрейт аудиозвонка (верхний график, пик ~1.25 Мбит/с) оказался выше и стабильнее, чем битрейт видеозвонка (нижний график, < 1 Мбит/с).**

Это объясняется работой адаптивных алгоритмов (QoS) приложения:

1. **Приоритет звука:** Для комфортного общения качество звука критичнее картинки, поэтому аудиопоток передается с высоким битрейтом.
2. **Адаптация видео:** Вероятно, из-за статической картинки в кадре (мало движения) или ограничений сети, алгоритм сжатия видео экстремально снизил битрейт видеопотока. Это доказывает, что "видео" не всегда означает "тяжелый трафик" — современные кодеки очень эффективно сжимают статичное изображение.

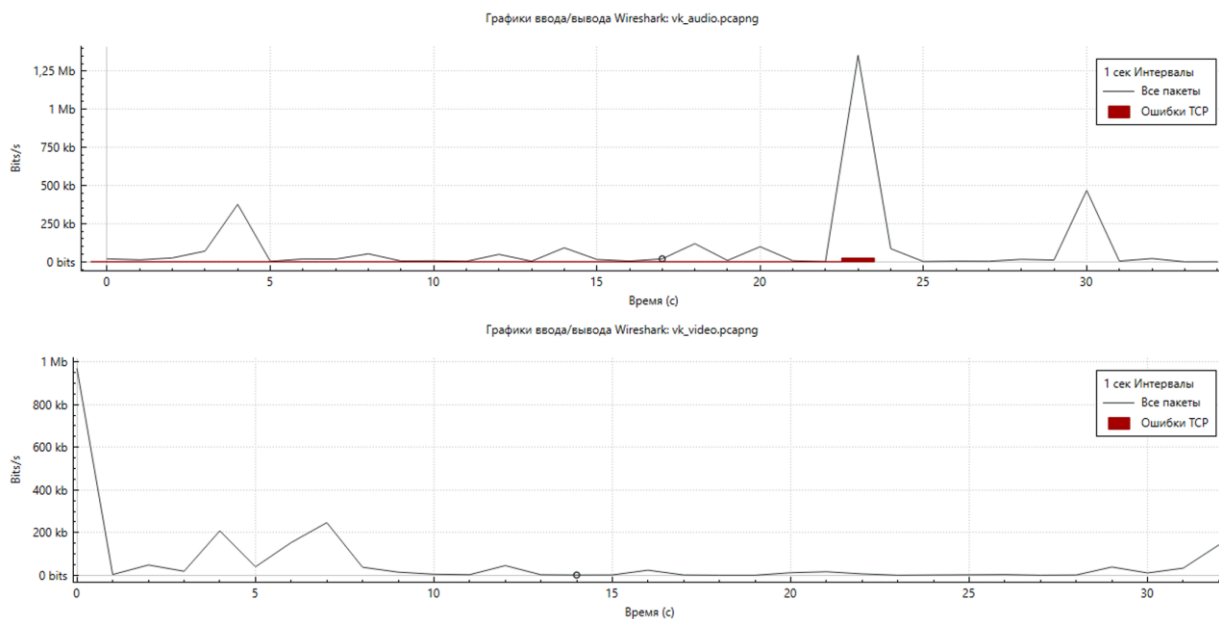


Рисунок 14 – Сравнительный анализ битрейта (I/O Graph) для аудио- и видеозвонков

2. Идентификация?

Для текстовых сообщений используется фильтр по TCP и IP-адресам серверов сообщений. Для медиапотокa эффективен фильтр по протоколу UDP или quic. Различение аудио и видео производится по анализу полосы пропускания (битрейта).

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы проведен детальный анализ сетевого трафика. Изучены на практике механизмы работы протоколов ICMP (фрагментация, TTL), принципы кэширования в HTTP, иерархия DNS, работа протоколов канального уровня (ARP) и особенности передачи данных в FTP и современных мессенджерах. Полученные навыки работы с Wireshark позволяют диагностировать сетевые соединения и анализировать структуру передаваемых данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Алиев Т.И., Соснин В.В., Шинкарук Д.Н. Компьютерные сети и телекоммуникации: задания и тесты. – СПб: Университет ИТМО – 2018. – 112 с.
2. Куроуз Дж. Ф., Росс К. В. Компьютерные сети: Нисходящий подход. – 6-е изд. – М.: Эксмо, 2016. – 912 с.